

AI / NLP Engineering Tasks & Terminology Workbook

Stage 4 Practice Guide for Junior Developers
Text Extraction · NLP · Scoring Logic · Matching

Hands-on tasks with key terms, adding intelligence to a backend
Builds directly toward Project 4: AI-Powered Resume Analyzer

How to Use This Workbook

This is the fourth workbook in the series. It assumes juniors are comfortable with Python core concepts (Stage 1) and can wrap logic behind an API (Stage 2). Stage 4 introduces **Natural Language Processing (NLP)** — teaching a program to read, clean, and make sense of unstructured text such as a resume.

Tasks progress from reading files, to cleaning text, to extracting meaning, to building a scoring system — exactly the pipeline Project 4 (Resume Analyzer) requires.

Each task is marked with a difficulty level so you can assign them based on a junior's pace:

Easy	Single library call or concept.	Medium	Combines extraction + logic.	Hard	Design and scoring judgment.
-------------	---------------------------------	---------------	------------------------------	-------------	------------------------------

Important: have juniors keep a small folder of 5–10 sample resumes (their own, templates, or anonymized examples) to test against throughout every section.

Prerequisite setup: Python with `pip install nltk spacy PyPDF2 python-docx` (or similar libraries), and the spaCy English model downloaded (`python -m spacy download en_core_web_sm`).

1 File Handling: Resume Upload

KEY TERMS

• file format (PDF/DOCX) • file validation • MIME type • upload endpoint • binary vs text file • error handling

#	Task	Level
1	Write a script that accepts a file path and checks whether it's a PDF or DOCX.	Easy
2	Reject and print a clear error for any other file type (e.g. .exe, .jpg).	Easy
3	Handle the case where the given file path doesn't exist.	Easy
4	Wrap file validation in a reusable function you can call from anywhere.	Medium
5	Build a simple Django endpoint that accepts a resume file upload.	Medium

2 Text Extraction

KEY TERMS

• text extraction • PyPDF2 / pdfplumber • python-docx • page • paragraph • raw text • encoding

#	Task	Level
1	Extract all text from a sample PDF resume using a PDF library.	Medium
2	Extract all text from a sample DOCX resume using python-docx.	Medium
3	Print the number of characters and words extracted from each file.	Easy
4	Handle a PDF that has no extractable text (e.g. a scanned image) gracefully.	Hard
5	Write one function <code>extract_text(file_path)</code> that works for both PDF and DOCX.	Medium

3 Text Preprocessing & Cleaning

KEY TERMS

• lowercasing • whitespace normalization • punctuation removal • stopwords • tokenization • lemmatization / stemming

#	Task	Level
1	Convert extracted resume text to lowercase and strip extra whitespace.	Easy
2	Remove punctuation and special symbols from the text.	Easy
3	Split the cleaned text into a list of words (tokenization).	Easy
4	Remove common stopwords (e.g. "the", "and", "is") using NLTK or spaCy.	Medium
5	Apply lemmatization so "managing" and "managed" both reduce to "manage".	Medium

4 Skill & Keyword Detection

KEY TERMS

• predefined skill list • keyword matching • named entity recognition (NER) • case-insensitive match • substring vs whole-word match

#	Task	Level
1	Create a list of 20–30 common tech skills (Python, Django, SQL, etc.).	Easy
2	Scan a resume's text and detect which skills from your list appear in it.	Medium
3	Make matching case-insensitive and avoid partial-word false matches (e.g. "Java" inside "JavaScript").	Medium
4	Use spaCy's NER to pull out entities like organizations or dates from a resume.	Hard
5	Return the detected skills as a clean list, not raw text.	Easy

5 Resume Scoring System

KEY TERMS

• scoring rubric • weighted score • score normalization • score out of 100 • consistency • edge case handling

#	Task	Level
1	Design a simple rubric: e.g. skills = 40%, experience = 30%, keywords = 30%.	Medium
2	Write a function that scores a resume based on how many required skills it contains.	Medium
3	Add points for years of experience mentioned in the text (basic pattern matching).	Hard
4	Normalize your final score so it always falls between 0 and 100.	Medium
5	Test your scoring function on 5 different resumes and check the scores make sense relative to each other.	Medium

6 Suggestions for Improvement

KEY TERMS

• feedback generation • missing skill detection • rule-based suggestion • actionable feedback • weak area

#	Task	Level
1	Compare a resume's detected skills against a target list and list what's missing.	Medium
2	Generate a suggestion like "Add more project experience" if the experience section is short.	Medium
3	Generate a suggestion about missing keywords relevant to a target role.	Medium
4	Keep every suggestion specific and actionable, not vague (avoid generic advice).	Medium
5	Return suggestions as a list so they're easy to display in an API response.	Easy

7 Job Description Matching

KEY TERMS

• job description parsing • similarity score • TF-IDF • cosine similarity • match percentage
• keyword overlap

#	Task	Level
1	Accept a job description as plain text input alongside a resume.	Easy
2	Extract keywords from the job description the same way you did for resumes.	Medium
3	Calculate a simple keyword-overlap match percentage between resume and job description.	Medium
4	Try TF-IDF + cosine similarity for a more advanced match score (optional stretch).	Hard
5	Display the match percentage alongside which required keywords were missing.	Medium

8 Ranking Multiple Resumes

KEY TERMS

• batch processing • sorting • ranking • comparative scoring • leaderboard

#	Task	Level
1	Process a folder of 5+ resumes and produce a score for each.	Medium
2	Sort and display resumes from highest to lowest score.	Easy
3	Rank resumes specifically against one job description (not just a general score).	Medium
4	Handle a broken or unreadable file in the batch without stopping the whole run.	Hard
5	Output the ranking as a clean table (name, score, match %, top missing skill).	Medium

9 Documentation, Testing & Delivery (toward Project 4)

This section maps directly to **Project 4 (AI-Powered Resume Analyzer)** deliverables. Completing these means a junior is ready to build the full project.

KEY TERMS

• sample dataset • output demonstration • scoring explanation • edge case testing • README • reproducibility

#	Task	Level
1	Build a small sample dataset of 5–10 test resumes (anonymized or template-based).	Easy
2	Test your system against an empty file and an unsupported file type.	Medium
3	Write a README section clearly explaining your scoring logic in plain language.	Medium
4	Capture sample input/output screenshots or logs for your README.	Easy
5	Commit your code, sample dataset, and README to GitHub with clear structure.	Easy

Tips for Mentoring on AI/NLP Work

- ✓ **Understand before importing** — juniors should know what a library call does, not just that it works.
- ✓ **Test on real-ish data** — clean, perfect text hides bugs that messy real resumes will expose.
- ✓ **Explain the scoring logic** — a score is only useful if you can justify why it's that number.
- ✓ **Handle edge cases early** — empty files and bad formats will happen; don't leave them for the end.
- ✓ **Start rule-based** — a solid rule-based system is a better foundation than jumping straight to ML.

Once juniors complete all nine sections, they should move on to Project 4 and then Stage 5 (Final Capstone) to combine everything into one deployable AI-enabled SaaS product.